Coding & Solution Implementation

CrediShield AI — Credit Card Approval Prediction System

---

1. Machine Learning Implementation (`train.py`)

The `train.py` script encapsulates the entire machine learning lifecycle from raw data ingestion to serialized artifacts.

1.1 Data Ingestion and Target Engineering

The pipeline begins by loading two Kaggle datasets:

- `application_record.csv`: Demographic and financial data.
- `credit_record.csv`: Monthly repayment statuses.

Target Label Creation: The raw data lacks explicit "Approve/Reject" labels. The script engineers these labels by tracking payment delinquency:

- **Approved (1)**: Clients whose worst payment status code indicates payments are up-to-date or under 30 days overdue (codes: C, X, 0, 1).
- **Rejected (0)**: Clients who have been 60+ days overdue at any point (codes: 2, 3, 4, 5).

1.2 Data Preprocessing

- **Missing Values**: The `OCCUPATION_TYPE` column is filled with the string `'Unknown'`.
- **Outlier Filtering**: Extreme income values are removed using the Interquartile Range (IQR) method to prevent skewing the scaling logic.
- **Deduplication**: Duplicate application records are dropped.

1.3 Feature Engineering

Derived features are calculated to provide the model with richer context:

- `AGE`: Calculated from the `DAYS_BIRTH` offset.
- `YEARS_EMPLOYED`: Calculated from `DAYS_EMPLOYED`, mapping unemployed statuses to 0.
- `UNEMPLOYED`: A binary flag (1/0).
- `INCOME_PER_MEMBER`: Total income divided by family size.
- **Categorical Buckets**: `INCOME_CAT` (Low/Medium/High), `AGE_CAT` (Young/Middle-Aged/Senior), and `EMPLOYED_CAT` (Unemployed/Junior/Mid/Senior).

1.4 Encoding and Scaling

- **Categorical Variables (11 columns)**: Processed using scikit-learn's `LabelEncoder` to transform text into integer representations.
- **Numerical Variables (6 columns)**: Processed using `StandardScaler` to ensure mean=0 and variance=1.
- Both the encoders and scaler are saved using `joblib` for deployment.

1.5 Model Training and Evaluation

The script trains 6 distinct algorithms using an 80/20 stratified split to preserve the extreme class imbalance ratio (~98% Approved / ~2% Rejected).

Performance Comparison:

Algorithm	Accuracy	F1-Score	ROC AUC
Logistic Regression	91.45%	95.53%	93.11%
Decision Tree	95.73%	97.69%	95.56%
Random Forest	97.85%	98.86%	96.98%
Gradient Boosting	**98.29%**	**99.14%**	**97.53%**
XGBoost	97.91%	98.91%	96.50%
Balanced Random Forest	86.12%	86.93%	92.84%

Selection: The pipeline automatically selects the **Gradient Boosting Classifier** as the winner due to its superior F1-Score, meaning it successfully balances precision and recall on the highly imbalanced dataset.

---

2. Backend Implementation (`app.py`)

The Flask application acts as the RESTful bridge between the user interface and the ML engine.

2.1 Initialization

On startup, the server loads the serialized `card_model.joblib`, `scaler.joblib`, `label_encoders.joblib`, and `feature_names.joblib`. If these files are absent, it programmatically invokes `train.py` to regenerate them.

2.2 Endpoint: `/predict` (POST)

Workflow:

1. **Validation**: Checks that the JSON payload contains all 14 required fields. Returns HTTP 400 if missing.
2. **Parsing**: Casts strings to integers and floats.
3. **Feature Engineering**: Recalculates the derived features (e.g., `income_per_member`, categories) exactly as done during training.
4. **Encoding**: Applies the saved `LabelEncoder`s to the categorical inputs.
5. **Scaling**: Passes the numerical inputs as a Pandas DataFrame to the saved `StandardScaler`.
6. **Assembly**: Constructs the final 18-element feature array by iterating over `feature_names.joblib` to guarantee identical column ordering.
7. **Inference**: Calls `model.predict()` to determine approval status, and `model.predict_proba()` to compute a confidence percentage.
8. **Risk Grading**: Assigns Low (≥85%), Medium (65-84%), or High (<65%) risk based on the confidence probability.
9. **Response**: Returns a JSON object: `{"approved": true, "confidence": 98.76, "risk_level": "Low"}`.

2.3 Endpoint: `/metrics` (GET)

Reads `metrics.json` and `stats.json` from disk and serves them to the frontend to power the Chart.js visualizations.

2.4 Server Configuration

Configured to bind to `0.0.0.0` and utilize the `PORT` environment variable to ensure compatibility with Render.com's cloud infrastructure.

---

3. Frontend Implementation

The frontend is a dynamic Single Page Application (SPA) driven by Vanilla JavaScript and CSS3.

3.1 Styling (`style.css`)

- **Glassmorphism**: Achieved using `backdrop-filter: blur(25px)` combined with semi-transparent background colors (`rgba(10, 15, 30, 0.65)`) over animated gradient blobs.
- **Theme System**: Utilizes CSS custom properties (variables) tied to a `[data-theme]` attribute on the root element, enabling seamless dark/light mode toggling.

3.2 Logic (`script.js`)

- **Form Submission**: Prevents default browser refresh, displays a loading spinner, and sends an asynchronous `fetch()` POST request to the API.
- **Result Rendering**: Parses the JSON response and updates the UI.
- **SVG Animation**: Animates the circular confidence gauge by mathematically calculating the `stroke-dashoffset` relative to the circumference ($C = 2 \pi r \approx 314.159$). Uses a `setInterval` loop to increment the numeric display in sync with the stroke animation.
- **Analytics Rendering**: Fetches data from `/metrics` and instantiates 4 separate Chart.js objects (bar, doughnut, pie) on HTML `<canvas>` elements.
- **Local Storage**: Pushes prediction results into a JSON array in the browser's `localStorage`, dynamically rendering an HTML table to display the history.
- **PDF Generation**: Generates an HTML string containing the "CrediShield AI Credit Assessment Letter," injects it into a hidden `<iframe>`, triggers `iframe.contentWindow.print()`, and cleans up the DOM.

Project Name: CrediShield AI
Author: Dinesh (dineshTech-creator)
Date: July 2026